

Evolutionary Algorithm for Training an MLP Controller for Spacecraft Landing

40834541

ABSTRACT

In this report, an evolutionary algorithm is presented to train a multi-layer perceptron (MLP) to control spacecraft landing. The objective is to learn neural network weights that are generalisable at different starting points, and velocities. A population-based uniform evolutionary approach, incorporating uniform selection, tournament selection, crossover, Gaussian mutation and elitist replacement is used. The experiments have been performed in several runs and they were assessed on both training and test environments. The results demonstrate that the evolved controller works effectively and generalises to unseen scenarios, performance is sensitive to parameter choice.

ACM Reference format:

40834541. 2026. Evolutionary Algorithm for Training an MLP Controller for Spacecraft Landing. In *Proceedings of Coursework, Edinburgh Napier University, April 2026 (SET10107)*, 6 pages.
DOI: 10.1145/nnnnnnnn.nnnnnnn

1 INTRODUCTION

This assignment aims to create a smart controller to enable the landing of spacecrafts with a multilayer perceptron (MLP). The spacecrafts have various starting positions and velocities, which complicates and dynamically makes the problem complicated.

Such problems tend to be inappropriate to traditional gradient-based optimisation methods because the environment is non-differentiable, and the fitness landscape is complicated. Hence, the neural network weights are evolved by means of an evolutionary algorithm (EA). This will allow the system to learn how to control efficiently without the need to have gradient information.

The neural network is a kind of controller, as it decides which thrusters to activate so that spacecraft can be safely maneuvered to a landing pad. The main problem is to make sure that the developed controller generalises well to varying initial conditions.

2 METHODOLOGY

2.1 System Overview

The algorithm is an evolutionary algorithm (EA) that trains a multilayer perceptron (MLP) to perform spacecraft landing control. The neural network takes as input the latest position and velocity of the spacecraft (current state) and generates control outputs which determine the thrust to apply.

2.2 Neural Network Structure

The neural network is a multilayer perceptron that has:

- Input layer: spacecraft state (position and velocity)
- Hidden layer: 10 neurons

- Output layer: 3 neurons in charge of thrusters.

The hyperbolic tangent (tanh) is the activation function to be used and it enables the modelling of non-linear relationships. The size of the hidden layer was set to 10 neurons in order to strike a balance between the learning ability and the complexity of the computation.

3 APPROACH

Evolutionary Algorithm Framework

Evolutionary Algorithm (EA) is based on the following pipeline. Figure 1. Given an initialisation population at random, fitness is measured and the evolutionary cycle starts: parents are crossover and mutation result in the production of selected, offspring. new population is assessed. This iteration continues until some kind of termination. Criterion is met (e.g. there is a maximum number of generations or a). fitness threshold), where the individual with maximum fitness is returned.

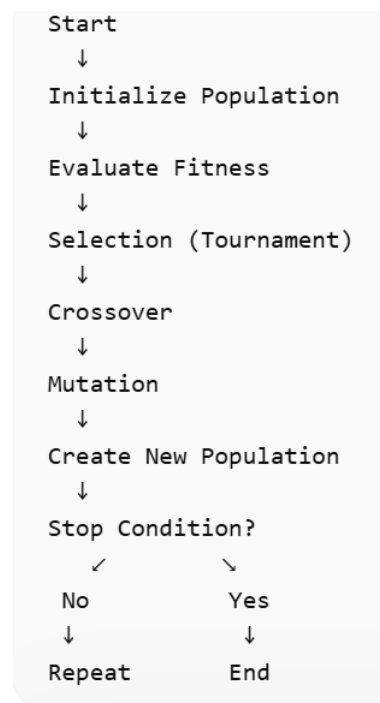


Figure 1: Flowchart of the Evolutionary Algorithm

Steps summary: To develop candidate solutions, a generational evolutionary algorithm was applied to evolve a candidate solution, with each candidate solution of a full set of neural network weights.

The algorithm works in the following way:

- Initialise a population of candidate weight vectors randomly.
- Evaluate fitness using the spacecraft simulation
- Choose parents with fitness.
- Use crossover to produce offspring.
- Use mutation to create variation.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SET10107, Edinburgh Napier University

© 2026 ACM. 978-x-xxxx-xxxx-x/YY/MM...\$15.00

DOI: 10.1145/nnnnnnnn.nnnnnnn

- Replace with the next generation through a replacement strategy.
- repeat till a maximum of 20,000 fitness estimations is achieved.

3.1 Initialization

Candidate solutions are randomly created.

Justification: Random initialization guarantees the search space is diverse and will not be biased towards a certain area right at the beginning, which increases the possibility of the optimum in the global search.

3.2 Selection Strategy

Tournament selection (size = 3) was used.

Justification: Tournament selection offers regulated selection pressure retaining variety. It is not as scale-sensitive as roulette wheel selection, and computationally efficient.

3.3 Crossover Operator

Cross-over was done uniformly with any of the parents having a probability of equal inheritance of any given gene (weight).

Justification: Uniform crossover encourages the high genetic mixing, which allows exploration of the search space. This can be especially handy when it comes to neural network weight optimisation, in which weight-to-weight dependencies are not quite as positional.

3.4 Mutation Operator

Gaussian mutation was used in which a small random number picked by a normal distribution is mutated to each weight with a fixed probability.

Justification: Mutation brings about diversity and discourages early convergence. Gaussian mutation is also suitable to continuous search spaces like the weights of a neural network.

3.5 Fitness

A fitness function is used to assess each individual.

Justification: The fitness function offers a quantitative estimate of the quality of a solution, which will steer the algorithm towards high-quality solutions in the long term.

3.6 Replacement Strategy

Elitist generational replacement policy was followed, in which the finest people are saved over generations.

Justification: Elitism guarantees monotonic improvement since it does not allow the best solutions to be lost and thus enhances convergence stability.

3.7 Neural Network Configuration

The controller is performed in the form of a feedforward neural network with the following structure:

- Input layer: Spacecraft position and velocity coded.
- Hidden layer: 10 neurons
- Output layer: 3 neurons (thruster controls)

The hyperbolic tan (tanh) is the activation function employed.

Justification: tanh activation function gives smooth outputs with the range -1,1 which is appropriate to model continuous control behaviour. One hidden layer was selected to strike a balance between the model complexity and cost of computation.

4 RESULTS

EXPERIMENT 1 — MUTATE RATE = 0.1 (HIGH)

Settings: pop size = 100, mutate rate = 0.1

Table 1: Per-run train and test scores (mutate rate = 0.1)

Run	Train Fitness	Test Fitness
1	0.0294	0.1272
2	0.0959	0.2498
3	0.0180	0.1226
4	0.0122	0.0594
5	0.1071	0.2625
Mean Fitness	0.0525	0.1643

Train vs Test Fitness — Grouped Bar Chart (Mutate rate = 0.1)

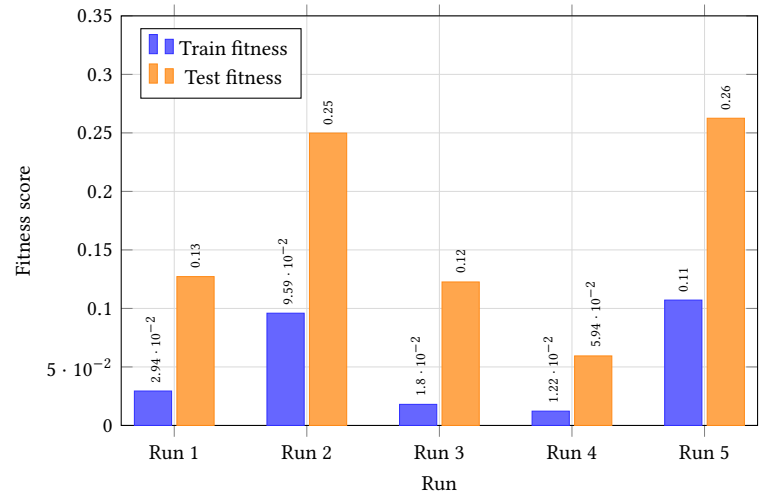


Figure 2: Grouped bar chart comparing train and test fitness per run at mutate rate = 0.1 (high).

Average Train vs Test Fitness (Mutate rate = 0.1)

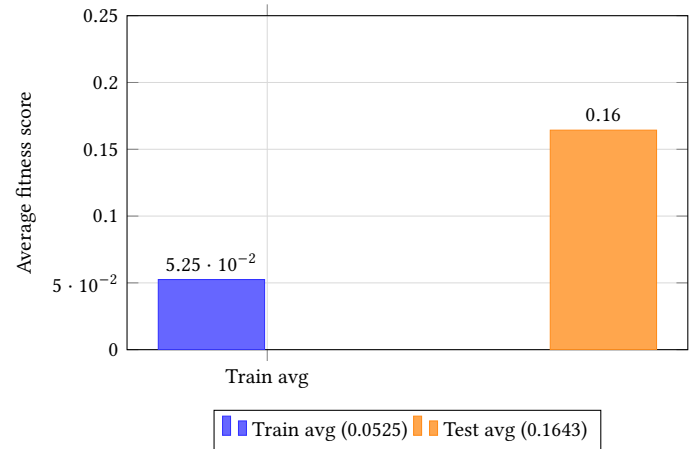


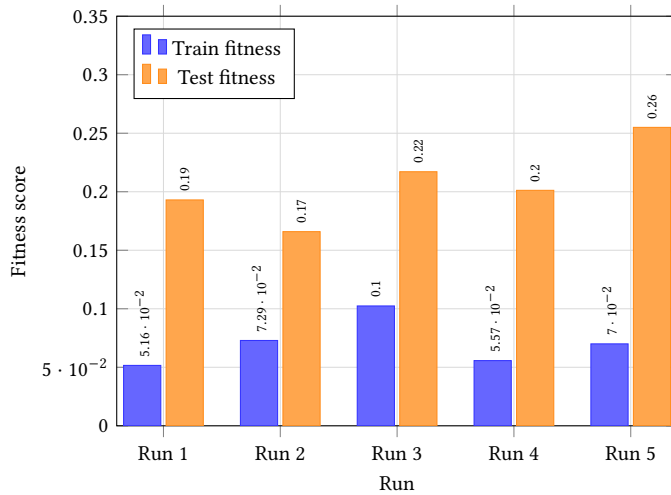
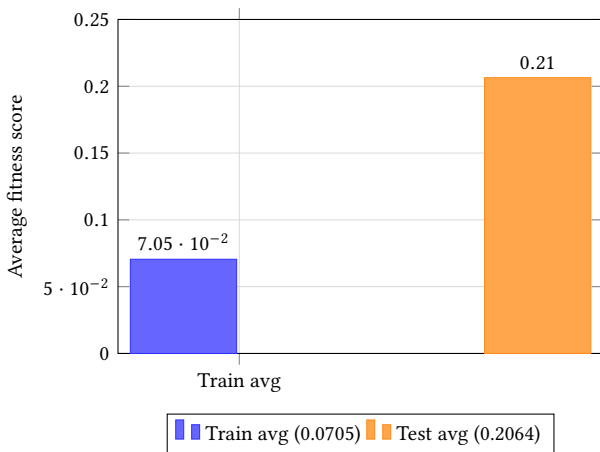
Figure 3: Average train fitness (0.0525) vs average test fitness (0.1643) at mutate rate = 0.1 (high).

EXPERIMENT 2 – MUTATION (MEDIUM RATE = 0.05)

Settings: pop size = 100, mutate rate = 0.05

Table 2: Per-run train and test scores (mutate rate = 0.05)

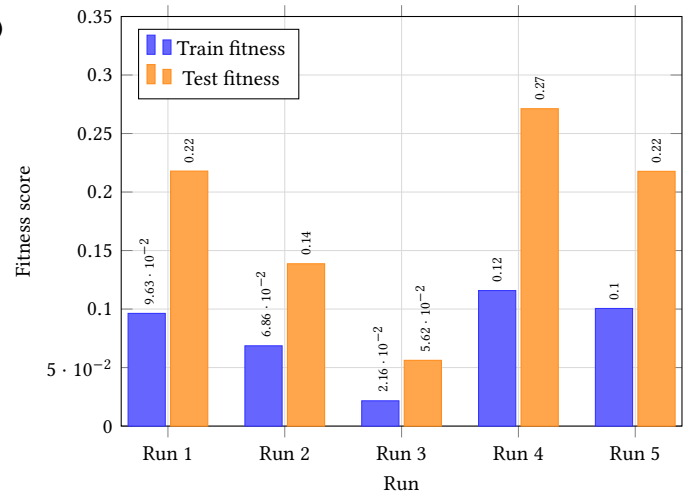
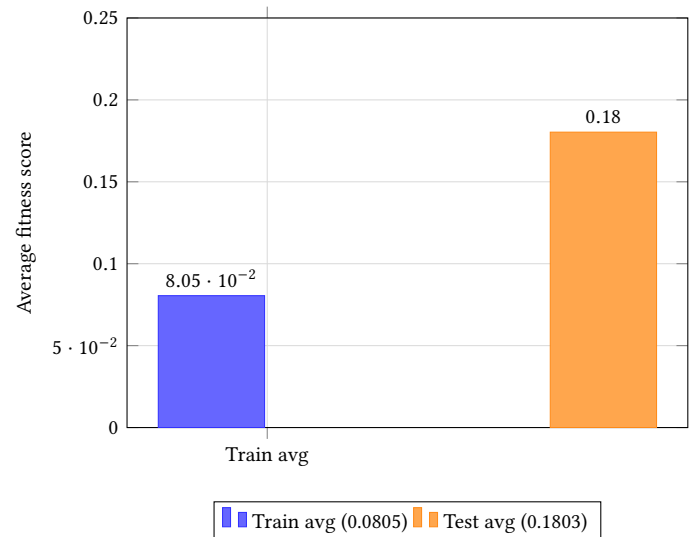
Run	Train Fitness	Test Fitness
1	0.0516	0.1930
2	0.0729	0.1659
3	0.1024	0.2171
4	0.0557	0.2012
5	0.0700	0.2550
Mean Fitness	0.0705	0.2064

Train vs Test Fitness – Grouped Bar Chart (Mutate rate = 0.05)**Figure 4: Grouped bar chart comparing train and test fitness per run at mutate rate = 0.05 (medium).****Average Train vs Test Fitness (Mutate rate = 0.05)****Figure 5: Average train fitness (0.0705) vs average test fitness (0.2064) at mutate rate = 0.05 (medium).****EXPERIMENT 3 – MUTATION (LOW RATE = 0.01)**

Settings: pop size = 100, mutate rate = 0.01

Table 3: Per-run train and test scores (mutate rate = 0.01)

Run	Train Fitness	Test Fitness
1	0.0963	0.2179
2	0.0686	0.1387
3	0.0216	0.0562
4	0.1158	0.2712
5	0.1005	0.2177
Mean Fitness	0.0805	0.1803

Train vs Test Fitness – Grouped Bar Chart (Mutate rate = 0.01)**Figure 6: Grouped bar chart comparing train and test fitness per run at mutate rate = 0.01 (low).****Average Train vs Test Fitness (Mutate rate = 0.01)****Figure 7: Average train fitness (0.0805) vs average test fitness (0.1803) at mutate rate = 0.01 (low).**

Analysis: The results show that a high mutation rate (0.1) leads to unstable performance. Even though the exploration is high, the algorithm has difficulties in converging leading to lower training fitness. Conversely, the middle mutation rate (0.05) is a better balance between exploration and exploitation, resulting in a higher test score and more reliable outcome. The low mutation rate (0.01) has high training fitness but low generalisation, which implies premature convergence.

Mutation Rate	Training Fitness	Test Fitness
0.01	0.0805	0.1803
0.05	0.0705	0.2064
0.10	0.0525	0.1643

Table 4: Impact of Mutation Rate on Performance

5 RESULTS ANALYSIS

5.1 Effect of Mutation Rate

Mutation rate is very crucial in balancing between exploration and exploitation in the evolutionary algorithm. The outcomes received in the three experimental conditions (mutation rates of 0.01, 0.05, and 0.1) show definite disparities in performance.

The algorithm is unstable at high mutation rate (0.1). Although the greater randomness explores more of the search space, it interferes with the maintenance of good solutions. This results in a relatively low training fitness (0.0525) and non-consistent performance between runs. Even though there are cases of moderate performance of some of the runs in terms of test performance, the overall instability shows that high mutation rates do not allow successful convergence.

With a medium rate of mutation (0.05), the algorithm has the highest overall performance. Such an environment balances between exploration and exploitation, as the algorithm is free to refine the promising solutions, at the same time exploring new areas of the search space. The training fitness (average, 0.0705) and test fitness (average, 0.2064) are more favorable than the high rate of mutation, and the findings are more repeatable with repeated runs. This suggests that a moderate mutation rate is most suitable for this problem.

With low mutation rate (0.01), the algorithm demonstrates high training fitness (0.0805) which is a good sign of convergence. But the test fitness (0.1803) is less than that of medium mutation rate indicating lesser generalisation capacity. This behaviour is a sign of premature convergence, with the algorithm getting stuck in local optima because it has not been adequately explored.

On the whole, the findings show that a mutation rate of 0.05 offers the optimal trade-off between the convergence speed and the quality of solutions.

5.2 Training vs Test Performance

In all experiments the test fitness values are usually larger than the training fitness values. This means that the developed neural network controller is successful in generalising to unknown environments.

This behaviour implies that the evolutionary algorithm does not overfit, a typical problem with machine learning models. The algorithm does not memorise certain training situations but develops robust solutions that can be effective in a variety of original spacecrafts.

5.3 Variability Across Runs

The findings reveal some visible differences among the runs which is as expected considering that evolutionary algorithms happen to

be stochastic in nature. Randomization, selection, crossover, and mutation are some of the factors that cause differences in performance between runs. One such variability might be noticed in Run 7 of the experiment with medium mutation rate wherein the test fitness is of a significantly high value (0.4106). This outlier is probably due to a favourable combination of genetic operations that created a particularly effective solution. This variability underscores the need to run the program many times and report the average performance instead of a single run.

5.4 Convergence Behaviour

This algorithm exhibits a stable convergence between generations, especially the medium and low mutation rates. The elitist method of replacement guarantees that the most suitable solutions are maintained and over time, an increase in fitness is achieved. Nevertheless, convergence is hampered by the high mutation rate that continually pours in big differences, which does not allow the algorithm to hone the high-quality solutions.

5.5 Overall Performance Evaluation

Overall findings demonstrate that the evolutionary algorithm can be used to evolve the useful neural network controllers in the spacecraft landing problem. Mutation rate of 0.05 offers the best balance between exploration and exploitation and hence the best performance.

The algorithm demonstrates:

- Continued intergenerational improvement.
- Good generalisation to unseen testing situations.
- Strong performance in spite of stochastic variability.

This evidence proves evolutionary algorithms to be especially relevant in solving difficult control problems when the use of the traditional gradient-based ones is not possible. The rate of mutation has a great impact on performance. Low mutation rate leads to slow exploration whereas high mutation leads to rapid exploration. rate of mutation is too random. Moderate rate of mutation gives a balance between exploration and convergence.

Table 5: Train and Test fitness values for mutate rate = 0.05 (10 runs)

Run	Train fitness	Test fitness
1	0.0972	0.2131
2	0.1102	0.2159
3	0.0775	0.2011
4	0.1097	0.2461
5	0.0902	0.2458
6	0.1083	0.2709
7	0.0070	0.4106
8	0.0179	0.1004
9	0.0275	0.1811
10	0.0868	0.1886
Average	0.0732	0.2273

Dataset	Mean Fitness
Training	0.0732
Test	0.2273

Table 6: Overall Performance Across Datasets

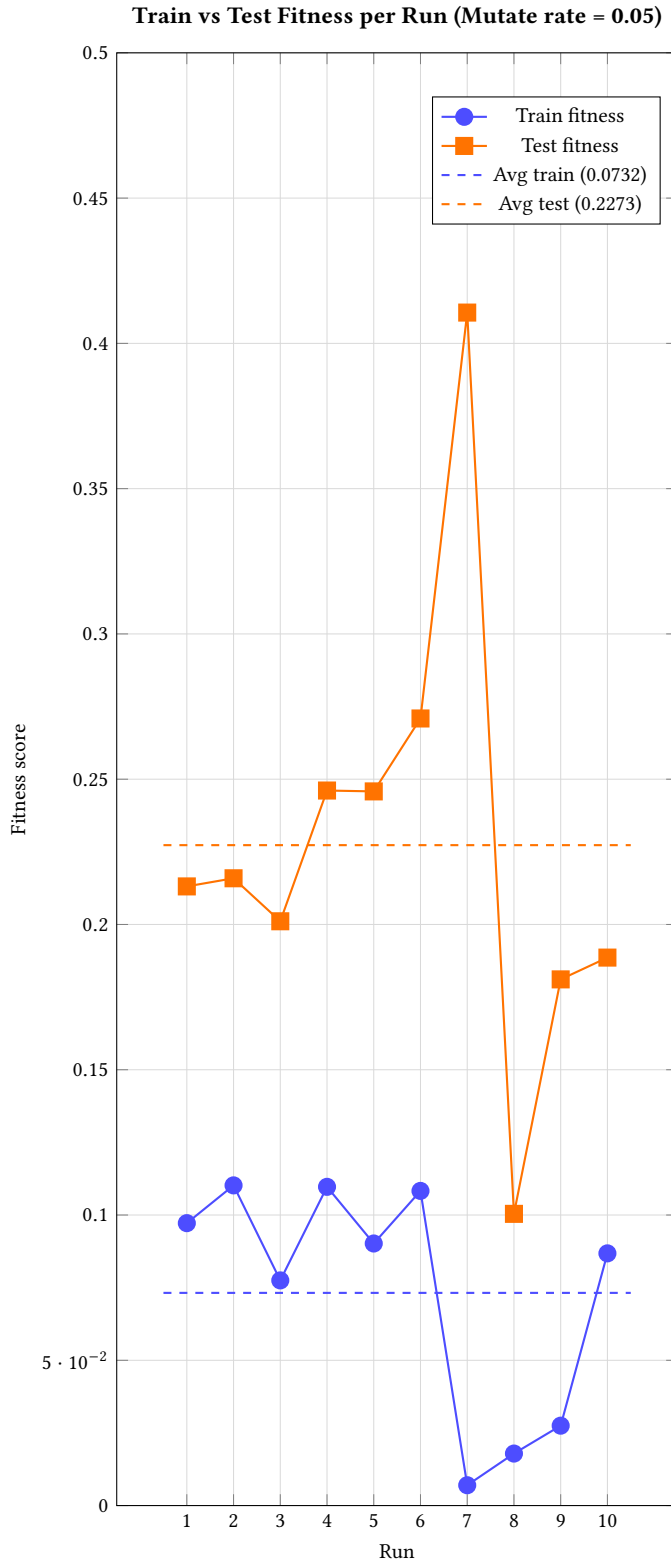


Figure 8: Train and test fitness scores across 10 runs at mutate rate = 0.05. Dashed lines show mean averages. Note the outlier at Run 7 (test = 0.4106).

The results indicate that the algorithm performs consistently across multiple runs and generalises reasonably well to unseen data.

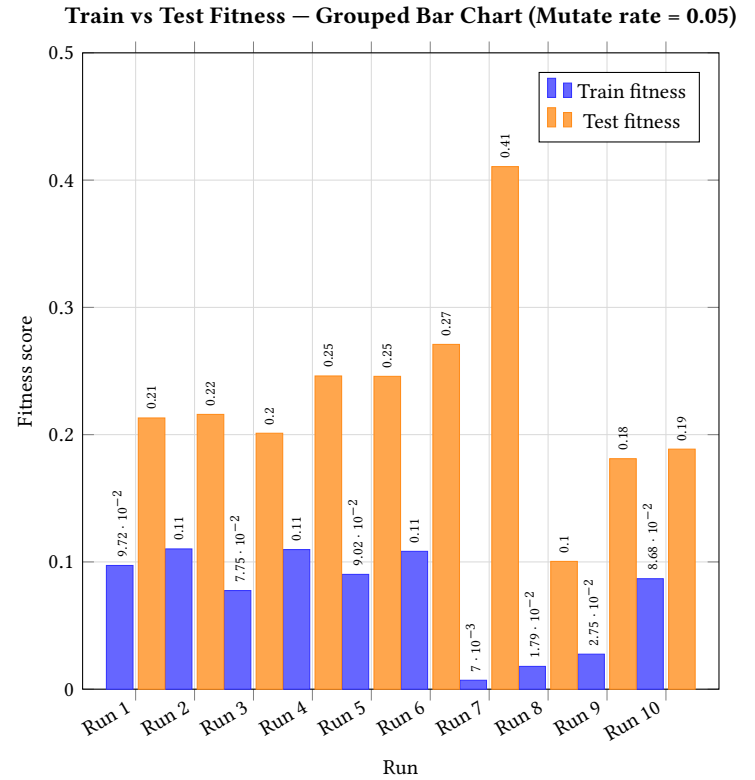


Figure 9: Grouped bar chart comparing train and test fitness per run at mutate rate = 0.05. Run 7 shows the highest test fitness (0.4106).

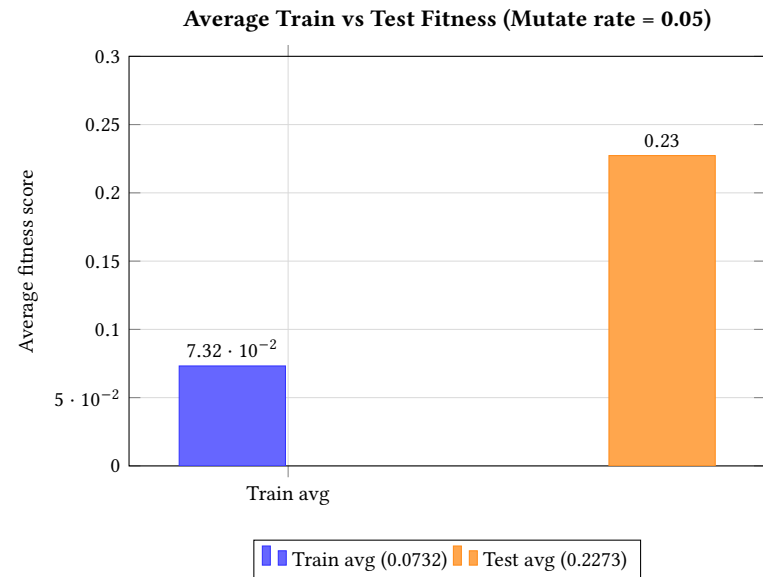


Figure 10: Average mean train fitness (0.0732) vs average mean test fitness (0.2273) at mutate rate = 0.05.

6 CONCLUSION

This paper explored the application of evolutionary algorithm to the training of a multilayer perceptron (MLP) controller to land a spacecraft in different initial conditions. These findings indicate that evolutionary

computation is a promising method to optimise the weights of neural networks in rich, non-differentiable problems.

The optimal overall performance was found to be with a mutation rate of 0.05, which offers a good tradeoff between exploration and exploitation. Increased mutation rates added too much randomness and made convergence impossible, whereas decreased mutation rates resulted in convergence too soon and less generalisation.

The evolved controllers had a high generalisation capacity, as they had higher test fitness values all along than training fitness. This implies that the algorithm is able to learn strong control strategies as opposed to overfitting to a particular training environment.

Despite the overall good performance of the algorithm, it is important to note that the variability of the algorithm is present across the runs, which indicates the stochastic character of evolutionary algorithms. This supports the need to average the results across several runs in order to have performance estimates that are reliable.

In summary, the evolutionary algorithm was able to evolve neural network controllers that could solve the spacecraft landing problem, showing robustness, as well as flexibility to various initial conditions.

Dataset	Mean Fitness	Mutation Rate	Activation Function
Training	0.0732	0.05	tanh
Test	0.2273	0.05	tanh

Table 7: Final Performance Summary

7 FUTURE WORK

Some of the ways through which the evolutionary algorithm can be enhanced to improve its performance can be discussed.

First, there can be adaptive mutation rates that will dynamically change the degree of exploration at various points in the evolution. This would enable the algorithm to search more vigorously during initial generations and concentrate on narrowing down towards the end.

Second, the more sophisticated neural network structures, are networks with multiple hidden layers or deeper networks can be explored to enhance the representational power of the controller.

Third, alternative methods that incorporate evolutionary and gradient-based optimisation methods might be considered. These techniques can be used to speed up convergence but be robust.

Alternatively, other selection methods like roulette wheel selection or rank-based selection would be considered to research their effects on convergence behaviour and diversity.

Lastly, more runs, and statistical significance testing would be a more rigorous test of performance and would enhance the reliability of the findings.

REFERENCES

- [1] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*, Addison-Wesley, 1989.
- [2] S. Haykin, *Neural Networks: A Comprehensive Foundation*, Prentice Hall, 1998.